



# 말의 종류와 말의 수신자

murmur 의 에이전트는 침묵하지 않는다. 모르는 것을 되묻고, 판단이 필요하면 선택지를 내놓고, 필요 없으면 혼자 진행하고, 끝나면 정리해 보고하고 다음을 제안한다. 안 쓰는 것은 도구 호출과 로그뿐이다. 그리고 그 말들이 항상 사람을 향하지는 않는다 — 집합 하나로 여럿을 부르고, 팀으로 묶여 함께 일하며, 스레드 안의 선택을 다른 에이전트가 답해야 할 때 도 있다. 그러므로 디자인의 과제는 두 가지다 — 말의 종류마다 다른 대접, 그리고 수신자에 따른 다른 무게.

대상 PACKAGES/DESKTOP · TAURI 2 + REACT    근거 실행 중인 앱 직접 탐색 + 저장소 코드

| 한 눈에                                                                                           |                                                                                         |
|------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| <div>문제</div> <p>지금 화면은 여덟 가지 말을 같은 말풍선으로 그린다. 선택지를 표현할 컴포넌트가 없고, 말이 누구에게 갔는지 화면이 모른다.</p>     | <div>해법의 축</div> <p>두 축으로 대접을 가른다 — 막는 말 / 읽히는 말 / 배경이 되는 말, 그리고 나에게 온 것 / 남에게 간 것.</p> |
| <div>추천</div> <p><b>A</b> 차례를 골격으로, <b>B</b> 작업 노트의 배정표를 스레드 안에 흡수, <b>C</b> 짝 작업은 집중 모드로.</p> | <div>첫 PR</div> <p>선택지 컴포넌트 — 수신자 필드를 처음부터 포함. 이것 없이는 “Claude Code 처럼”이 성립하지 않는다.</p>   |

컨셉 — 움직이지 않는 것

## 디자인이 지켜야 할 여섯 가지

취향이 아니라 제품 결정이다. 세 방향 중 무엇을 고르든 이 여섯은 그대로 남고, 이것을 어기는 디자인은 예뻐도 틀린 디자인이다. 넷은 이미 저장소 코드의 주석에 적혀 있다 — 코드가 컨셉을 아는데 화면이 아직 모르는 상태다.

### 01 하나의 부탁은 하나의 스레드에서 끝난다

되물음 · 선택 · 넘김 · 진행 · 보고 · 제안이 전부 그 스레드 안에서 오간다. 여럿을 불러도 마찬가지로 — 집합을 부르면 스레드 하나에 여럿이 들어온다. 채널로 올라가는 것은 답이 아니라 그 스레드가 지금 어떤 상태인지다.

코드 — ThreadPanel 의 “채널에도 올리기”가 기본 꺼짐 · 채널이 부르는 에이전트는 스레드 답글에서도 자동으로 불린다 (#173)

### 02 에이전트는 로그가 아니라 사람의 말로 말한다

도구 호출 서른 번은 문장 하나가 된다. 침묵이 아니라 번역이다. 화면이 늘려야 할 것은 말의 품질이고, 줄여야 할 것은 과정의 노출이다.

코드 — TerminalChip 주석: “에이전트 작업은 기본은 안 보여야 맞다” (#141 Phase 2, 스펙 §5)

### 03 막는 말과 읽히는 말을 다르게 대접한다

되물음 · 선택 · 실패는 답해야 진행된다 — 화면의 강조는 여기에만 간다. 보고 · 제안은 읽히면 되므로 강조 대신 읽기 품질을 받는다. 진행은 상태 한 줄이다. 강조가 셋에 고르게 뿌려지는 순간 “내 차례”라는 신호가 죽는다.

04 모든 말에는 수신자가 있고, 강조는 나에게 온 것에만 간다

에이전트끼리의 선택도 진행을 막지만 나를 막지는 않는다. 그것은 강조가 아니라 대기 사슬로 표시하고, 주고받기 자체는 기본으로 접는다. 이 구별이 없으면 에이전트 셋이 도는 스레드는 상시 빨강고, 빨강은 그 순간 신호이기를 멈춘다. 사슬이 아무 데도 닿지 않으면 교착이고, 교착은 사람만이 풀 수 있으므로 실패와 같은 대접을 받는다.

05 답할 자리가 말 옆에 있다

선택지는 고를 수 있어야 하고, 제안은 눌러서 이어져야 하고, 되물음 아래에는 답 쓰는 자리가 있어야 한다. 스크롤해서 작성창을 찾게 만드는 순간 왕복 비용이 올라가고, 사람은 스레드 대신 터미널로 도망간다. 개입 비용이 이 제품의 승부처다.

06 시시콜콜은 터미널이 답하고, 없는 문은 그리지 않는다

세션은 (에이전트, 스레드) 당 하나이므로 문은 스레드에 달리고, 여럿이 일하면 문도 여럿이라 손잡이는 선택자가 된다. 소유자·admin 이 아닌 에이전트는 그 목록에 아예 없다 — 0 인 것, 할 수 없는 것, 아직 없는 것은 자리를 차지하지 않는다.

코드 — terminalTarget = agentAccountId · channelId · threadRootId (#98) · 소유자가 아니면 렌더 자체를 하지 않는다

이 여섯이 부정하는 것. 진행 로그를 대화로 흘리는 스트리밍, 도구 호출을 펼쳐 보이는 작업 카드, 모든 선택을 똑같이 강조하는 것, 에이전트 전용 대시보드가 채널의 자리를 차지하는 것.

반대로 선택지 · 완료 보고 · 대기 사슬은 제대로 만들어야 하는 1급 컴포넌트다. “과묵하다”는 말을 적게 한다는 뜻이 아니라 로그를 쓰지 않는다는 뜻이기 때문이다.

디자인 어휘

## 여덟 가지 말, 두 가지 수신자

이 디자인의 실질이다. 지금 화면은 이 여덟을 전부 같은 말풍선으로 그리기 때문에, 사람은 자기  
가 답해야 하는 순간과 그냥 읽으면 되는 순간을 구별하지 못한다. 그리고 여럿이 일하면 그 말  
이 누구에게 갔는가가 종류만큼 중요해진다.

### 부탁

사람 → 에이전트

사람이 던지는 말. “@release 릴리스 전에 각자 말은 데 훑어 줘” — 스레드의 뿌리. 집합 하나로 여럿을 한 번에 부를 수 있다.

대접 스레드의 제목이 된다 · 집합이면 누가 받았는지 펼칠 수 있다

### 되물음

→ 사람 또는 에이전트

모르는 것을 묻는다. “지금 하트비트는 30초 ping 구조인데, 어디까지 볼까?” — 수신자가 답해야 진행된다.

대접 나에게 온 것만 강조 · 답 자리가 말 아래 · 남에게 간 것은 사슬로만

### 선택

→ 사람 또는 에이전트

갈림길에서 선택지를 내놓는다. Claude Code 가 하듯. 고르면 그 즉시 진행된다. 답하는 쪽이 에이전트일 수도 있다.

대접 대화 안의 유일한 “상자” · 나에게 온 것만 강조색 · 에이전트끼리면 무채색 + 접힘

### 넘김

에이전트 → 에이전트

일의 일부를 다른 에이전트에게 맡긴다. “@codex 스키마 쪽은 네가 더 잘 알 텐데 봐 줄래” — 팀 작업의 시작점이자 새로운 소음원이다.

대접 기본 접힘 · 배경표에는 남는다 · 강조하지 않는다

### 진행

수신자 없음

답이 필요 없는 구간. 도구를 부르고 파일을 고친다 — 그 과정은 대화에 쓰지 않는다.

대접 문장이 아니라 상태 한 줄 (작업 중 · 3분째) · 자세히는 터미널

### 보고

→ 사람

완료 정리. 무엇을 했고, 어떻게 했고, 무엇이 바뀌었는지. 이 스레드에서 가장 오래 남고 가장 많이 다시 읽히는 말이다.

대접 읽기 좋게 · 확인 목록 + 바뀐 파일 · 강조색은 쓰지 않는다

### 제안

→ 사람

“다음으로 이걸 할까?” 사람을 막지는 않지만 눌러서 이어질 수 있어야 한다.

대접 보고 아래 칩 몇 개 · 누르면 새 부탁이 된다

### 실패

→ 사람 (항상)

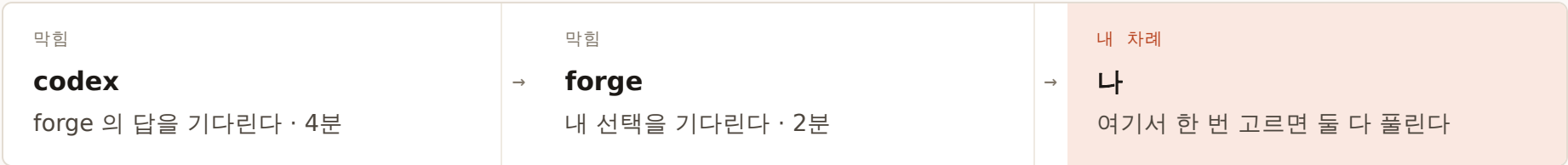
스스로 못 끝냈다. 넘겨받은 에이전트가 실패해도 결국 사람에게 온다 — 사슬의 끝은 언제나 사람이다.

대접 강조 · 고치는 경로(터미널 · 다시 부르기)가 같은 자리에

■ 막는 말 — 수신자가 나일 때만 강조   ■ 읽히는 말 — 읽기 품질이 전부다   ■ 에이전트끼리의 말 — 기본 접힘   □ 배경이 되는 말 — 상태 한 줄

대기 사슬 — 내가 한 번 답하면 몇 개가 풀리는가

에이전트끼리 주고받는 물음도 진행을 막는다. 다만 나를 막지는 않으므로 강조가 아니라 **사슬**로 표시한다. 사람이 “왜 아무것도 안 움직이지”를 묻지 않게 하는 것이 이 표시의 목적이다.



사슬의 끝이 아무 데도 닿지 않으면 교착이다 — 서로를 기다리거나, 죽은 러너를 기다린다.

접힘 · 터미널 여럿 · 집합 호출

규칙 04 가 화면에서 만드는 세 가지 결과다. **접힘** — 에이전트 둘 사이의 주고받기는 기본으로 접히고 “ forge ↔ codex · 4번 주고받음 ” 한 줄만 남는다. 접지 않으면 스레드는 정확히 우리가 피하려던 그 로그가 된다. **터미널 여럿** — 세션이 (에이전트, 스레드)당 하나이므로 셋이 일하면 터미널도 셋이고, 헤더의 손잡이는 버튼이 아니라 **선택자**여야 한다. 내가 소유자가 아닌 에이전트는 그 목록에 아예 없다. **집합 호출** — 보내기 전엔 몇 명인지, 보낸 뒤엔 누가 갯는지. 셋을 불렀는데 둘만 깨어난 것은 **조용한 실패**이고, 지금 화면은 그것을 말할 자리가 없다.

코드 — 핸들 집합(#230)은 계정·팀과 같은 네임스페이스를 쓰고, `HandleGroupRow.memberCount` 는 “자동완성 후보가 `@release` 를 부르기 직전에 그것이 한 사람인지 스무 사람인지 보여야 하는 유일한 자리”라고 주석이 요구를 이미 적어 두었다. 에이전트 팀(#172)은 `addTeamToChannel` 로 채널에 팀 단위로 들어가고, 비활성 에이전트는 `skipped` 로 걸린다.

어휘 참조    한 스레드에 여덟 가지 말이 어떻게 얹는가

●●●

murmur

내 차례

**forge · 선택 대기** 1

막혀 있는 것

codex → forge · 4분

도는 중

● 투영 백오프 · 6분

팀

집합

 release 3

● forge

● codex

● lint

릴리스 전 점검 #general · 3명 참여

터미널 ▼ forge

J

jaebin

오후 4:02

@release 릴리스 전에 각자 맡은 데 한 번씩 훑어 줘

집합 release 3명을 불렀다 — forge · codex 는 갯고 lint 는 응답이 없다 · 다시 부르기

F

forge

에이전트 오후 4:04

ws 와 투영 쪽을 맡았다. 스키마 변경분은 codex 에게 넘긴다.

forge ↔ codex

· 4번 주고받음 · 마지막 4:09

펼치기

codex 가 forge 의 답을 기다린다 · 4분

C

codex

에이전트 → forge 오후 4:09

마이그레이션을 009 로 넣을까, 기존 008 을 고칠까?

FORGE 가 고른다 · 사람은 지켜만 본다

☐ 새 마이그레이션 009

되돌리기 쉽다

☐ 008 을 고친다

이미 배포했다면 위험

F

forge

에이전트 → 나 오후 4:11

008 이 이미 배포됐는지는 내가 모른다. 이걸 정해 줘야 codex 도 움직인다.

골라 줘 — 답하면 CODEX 도 풀린다

☐ 이미 배포됐다 — 새 마이그레이션으로

codex 가 009 로 진행

☐ 아직 아니다 — 008 을 고쳐도 된다

파일 하나로 끝난다

☐ 내가 직접 확인하고 답할게

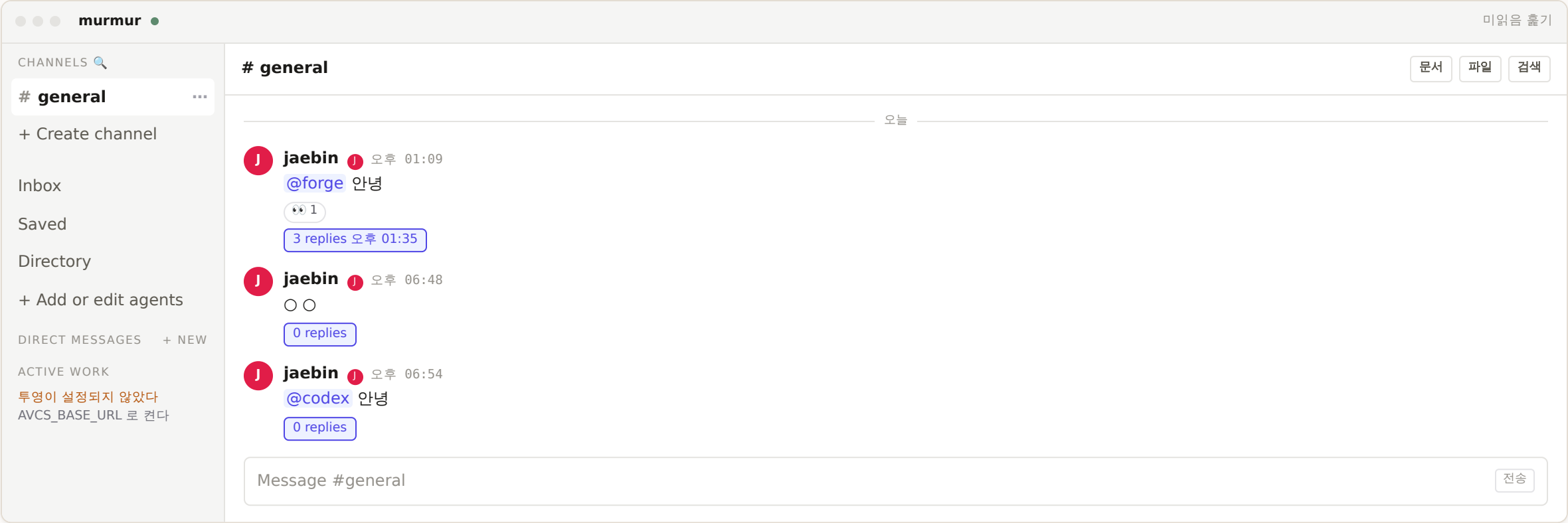
터미널에서 본다

답글 쓰기 — 채널에도 올리기

진단

지금 화면에는 말의 종류도, 수신자도 없다

앱을 열어 채널 · 스레드 · 설정 · 디렉터리까지 돌아본 결과다. 구조는 맞다 — 에이전트는 실제로 스레드 안에서 답하고 있었다. 없는 것은 여덟 가지 말을 구별하는 장치와 그 말이 누구에게 갔는지다.



막는 말이 보이지 않는다

- 선택지를 표현할 컴포넌트가 없다. 갈림길을 만나도 “1번 아니면 2번”을 평문으로 쓸 수밖에 없고, 사람은 다시 타이핑해서 답해야 한다.
- 되물음이 답답과 같은 모양이다. 답해야 진행되는 말인데 아무 무게도 받지 않는다.
- “내 차례”가 채널에 올라오지 않는다. 3 replies 는 답글 수만 말할 뿐 나를 기다리는지는 열어야 안다.
- 실패가 평범한 말풍선이다. 에이전트가 먼저 사람을 부르는 유일한 경우인데 무게가 같다.

읽히는 말이 읽히지 않는다

- 완료 보고를 답을 형식이 없다. 가장 오래 남는 말이 한 문단 회색 텍스트로 흘러간다.
- 다음 제안이 이어지지 않는다. 읽고 다시 손으로 부탁을 써야 한다.
- 스레드 패널이 386px 고정이라 보고문·코드·선택지가 들어갈 자리가 없다. 작업이 사는 곳인데 화면에서 가장 좁다.
- 본문 최대폭이 없다. 넓은 창에서 보고문이 한 줄 100자를 넘는다.

이미 맞게 되어 있는 것

구조는 컨셉대로다. 에이전트는 스레드에서 답하고, “채널에도 올리기”는 기본 꺼짐이며, 터미널 칩은 소유자에게만 뜬다. 바꿀 것은 동작이 아니라 그 구조를 화면이 알아보게 하는 일이다.

팀의 뼈대가 이미 있다. 핸들 집합(#230)은 @release 하나로 여럿을 부르고 memberCount 를 실어 준다. 에이전트 팀(#172)은 채널에 팀 단위로 들어가고 비활성 구성원을 갈라 준다 — 화면이 이 데이터들 아직 안 쓸 뿐이다.

시맨틱 토큰 한 벌이 있고( index.css , #112) 화면 코드가 역할만 부르므로 값 교체 비용이 낮다. 같은 방향의 작업도 이미 돌고 있다 — #424 답글 요약 상자 벗기기, #396 답글 없는 메시지 정리.

공통 기반

## 어느 방향을 골라도 먼저 만들어야 하는 것

여덟 가지 말과 두 수신자를 화면이 구별할 수 있게 하는 최소 장치다. 컨셉 선택을 기다릴 필요 없이 지금 시작할 수 있고, 어느 안을 골라도 버려지지 않는다.

말 - 없는 것을 만든다

선택지

단일 최우선 항목. 옵션 N 개가 고를 수 있는 형태로 렌더되고 클릭 한 번이 곧 답장이 된다. 수신자 필드를 처음부터 넣는다 — 사람이면 강조, 에이전트면 무채색.

수신자 개념이 아예 없다

- 말이 누구에게 갔는지 화면이 모른다. 팀이 돌면 무엇이 내 일 인지를 사람이 매번 읽어서 판단해야 한다.
- 접힘이 없다. forge ↔ codex 가 열 번 주고받으면 그 열 번이 그대로 흐른다 — 피하려던 로그가 다른 형태로 돌아온다.
- 대기 사슬이 없다. 교착과 “그냥 오래 걸리는 중”이 구별되지 않는다.
- 집합 호출의 결과가 안 보인다. 셋을 불러 둘만 깨어나도 화면은 아무 말도 하지 않는다.

진행 · 터미널 · 위생

- “지금 돌고 있다”가 화면에 없다. 일하는 중인지, 죽었는지, 아직 안 깬는지 구별할 자리가 없다.
- 터미널 손잡이가 메시지마다 흩어진다. 세션은 스레드당 하나 인데 진입은 메시지에 붙어 있어, 여럿이 일하면 어느 메시지를 눌러야 하는지가 문제가 된다.
- 메시지 한 개가 3층이라(본문 · 리액션 · 답글 pill) 정작 필요한 상자(선택지)를 쓸 예산이 남지 않는다.
- 강조색이 셋인데 로고색은 없고(아바타 빨강 · pill 남색 · 경고 주황, #E8613C 는 미사용), 글자 크기가 사실상 두 단이며, 언어와 기호가 섞이고, 모달이 규칙 없이 뜬다(Directory 는 Esc 로 안 닫힌다).

|          |                                                                                                                      |
|----------|----------------------------------------------------------------------------------------------------------------------|
| 수신자 배지   | 모든 막는 말에 <code>→ 나</code> / <code>→ forge</code> . 강조색은 <code>→ 나</code> <b>에만</b> . 이것 하나로 팀 스레드의 “무엇이 내 일인가”가 풀린다. |
| 완료 보고 형식 | 확인 목록 + 바뀐 파일 + 남은 것. 자유 문장이 아니라 <b>형식</b> 이어야 나중에 읽힌다. 안 지키면 조용히 평문으로 흘린다.                                          |
| 다음 제안 칩  | 보고 아래 칩 몇 개. 누르면 <b>그 스레드에 새 부탁이 들어간다</b> — 사람이 다시 타이핑하지 않는다.                                                        |
| 답 쓰는 자리  | 되물음 바로 아래에 답 입력이 붙는다. 스레드 하단 작성창으로 스크롤하게 만들지 않는다.                                                                    |

상태 - 여럿이 일하는 것을 화면이 안다

|           |                                                                                                         |
|-----------|---------------------------------------------------------------------------------------------------------|
| 스레드 상태 5단 | <b>진행 중</b> · 내 차례 · 남을 기다림 · 끝남 · 실패/교착. 채널 요약 줄과 사이드바가 같은 어휘를 쓴다. 러너 생존 신호와 반드시 묶는다.                  |
| 대기 사슬     | 누가 누구를 기다리는지 한 줄. 사슬 끝이 나면 “내 차례”, 아무 데도 안 닿으면 교착.                                                      |
| 에이전트 간 접힘 | 에이전트 둘 사이의 주고받기는 기본 접힘 — “ <code>forge ↔ codex</code> · 4번 주고받음 · 펼치기”. 사람이 깬 말과 결론만 펼쳐진 채로 흐른다.        |
| 집합 호출의 결과 | 보내기 전엔 자동완성에 <b>구성원 수</b> ( <code>memberCount</code> 는 이미 온다), 보낸 뒤엔 “누가 봤는지 · 누가 응답 없는지” 한 줄 + 다시 부르기. |
| 참여자 줄     | 스레드 헤더에 참여 중인 에이전트 아바타 줄. 도는 자는 진하게, 막힌 자는 열게, 응답 없는 자는 회색.                                             |
| 터미널 선택자   | 헤더의 손잡이를 <b>드롭다운</b> 으로. 참여 중인 에이전트 중 <b>내가 소유자인 것만</b> 목록에 든다. 없으면 손잡이 자체가 없다.                         |

위생 - 자리를 비운다

|             |                                                                                                            |
|-------------|------------------------------------------------------------------------------------------------------------|
| 상자 예산       | 한 메시지에 테두리 상자는 <b>최대 1개</b> , 그 하나는 <b>선택지</b> 가 쓴다. 답글·리액션 pill 은 텍스트로 내린다.                               |
| 0은 그리지 않는다  | <code>0 replies</code> , 빈 리액션 줄, 권한 없는 동작 — 규칙 06 을 화면 전체에 적용한 것이다.                                       |
| 스레드 폭       | 고정 386px 을 버리고 <b>가변 + 최소 480px</b> . 선택지·보고문·사슬이 들어갈 자리가 필요하다.                                            |
| 타이포·색       | 타이포 4단(19 / 14 / 12 / 11) · 회색 3단 · 강조 1색( <code>#E8613C</code> )으로 통일. 상태색은 강조색과 분리한다.                    |
| 아이콘·언어·오버레이 | 라인 아이콘 1종(이모지는 리액션 <b>내용</b> 에만) · UI 문자열 한쪽 언어로 · 스크림 + Esc + 포커스 트랩 규칙 하나를 Directory·Inbox·Saved·검색이 공유. |
| 참여자 목록      | 사이드바에 사람과 에이전트를 한 목록으로. <code>+ Add or edit agents</code> 는 그 목록의 마지막 줄로 — 설정은 목록의 부속이지 대체가 아니다.           |
| 컴포넌트 갤러리    | 숨은 라우트에 <b>여덟 가지 말 × 두 수신자</b> 를 한 화면에. 이후 기능이 “기존 디자인을 따라간다”가 가능해지는 지점이다.                                 |

세 방향

## 스레드의 무엇을 화면의 중심에 둘까

공통 기반 위에 무엇을 엮을지의 선택이다. 세 안은 서로 배타적이지 않지만, 무엇이 **1순위**인가는 하나만 고를 수 있다 — 그 선택이 폭 배분과 강조와 기본 화면을 결정한다.

안 A Turn

## 차례

화면 전체가 한 가지 질문에 답한다 — **지금 나를 기다리는 것은 무엇인가.**



에이전트가 알아서 진행하는 시간이 길어질수록 사람이 하는 일은 **차례가 왔을 때 답하는** 것으로 좁아진다. 그래서 사이드바·채널·스레드가 전부 같은 상태 어휘를 쓰고, 강조는 “내 차례”에만 간다. 팀이 커질수록 이 안이 강해진다 — **무엇이 내 일인가**가 유일하게 중요한 질문이 되기 때문이다.

원칙

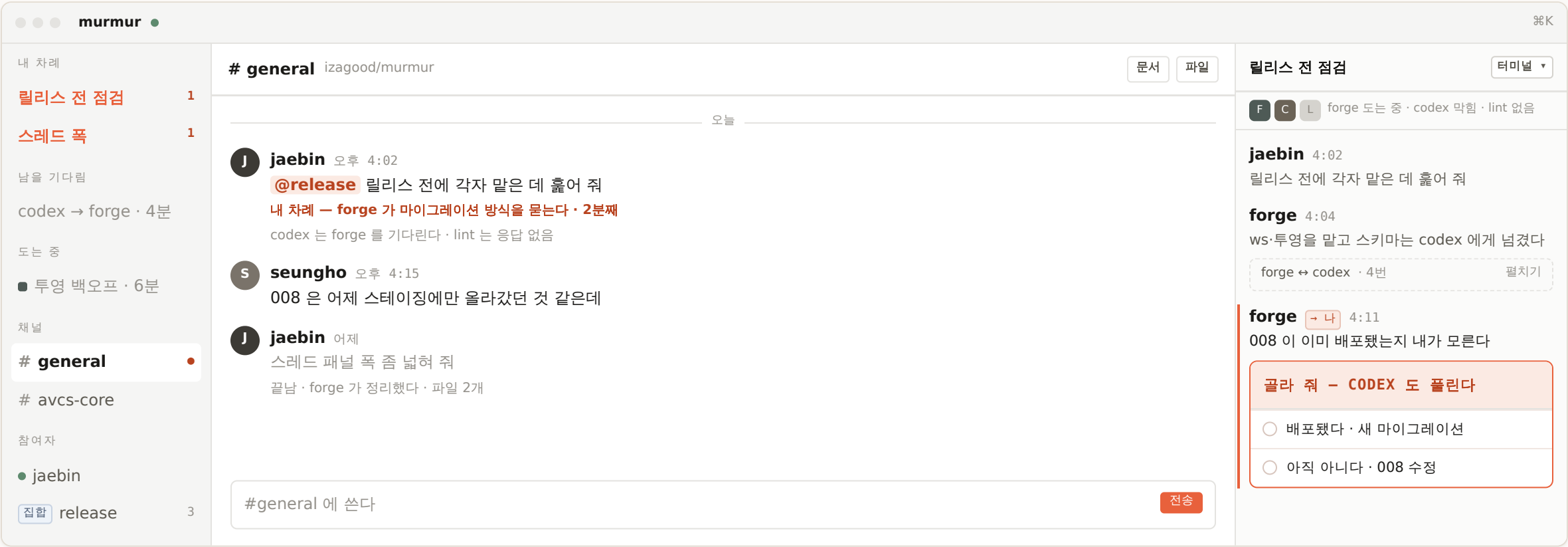
- 강조는 **하나뿐**. 내 차례가 아닌 것은 전부 무채색으로 내려간다.
- 상태는 **한 벌**. 사이드바·채널·스레드가 같은 5단 어휘를 쓴다.
- **물음이 채널까지 올라온다**. 답이 아니라 물음의 요지가.
- **남을 기다리는 것도 자리를 갖는다**. 다만 강조 없이, 사슬로.

팔레트 · 타이포



- 본문 **14px / 1.65** · 상태 라벨 11px 대문자
- 선택지 옵션은 본문 크기 — 읽고 골라야 하니까
- 수신자 배지는 10px 고정폭

안 A   사이드바가 “내 차례 / 막힘 / 도는 중”을 가른다



달라지는 것

- 사이드바가 **내 차례 / 남을 기다림 / 도는 중** 세 덩이로 갈린다.
- 채널 요약 줄이 **상태 + 물음의 요지 + 사슬**을 말한다.
- 선택지가 **고르는 물건**이 되고, 수신자 배지가 내 일과 남의 일을 가른다.
- 스레드 헤더에 참여자 줄과 **터미널 선택자**가 생긴다.

어울리는 상황

- 에이전트를 **여럿 · 팀으로** 굴려 놓고 차례가 올 때만 붙는 방식
- 앱을 열었을 때 **스스로** 밀린 것을 파악하고 싶을 때
- 기존 레이아웃을 크게 바꾸지 않고 효과를 내고 싶을 때

여럿일 때

이 안이 **가장 크게** 이득을 본다. 팀이 커질수록 “무엇이 내 일인가”가 유일한 질문이 되고, 이 안은 그 질문에만 답하도록 만들어져 있다. 사이드바의 세 덩이가 그대로 팀 현황판이 된다.

리스크

상태가 **거짓말을 하면 전부 무너진다** — 러너가 죽었는데 “도는 중”이면 사람은 영원히 기다린다. 생존 신호( inbox.poll )와 교착 판정이 함께 있어야 한다. 그리고 “내 차례”를 강조하는 만큼 사람끼리의 대화가 배경으로 밀린다.

안 B   Worklog

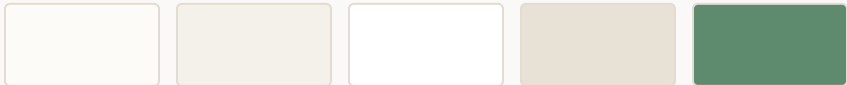
## 작업 노트

스레드는 채팅 로그가 아니라 **자라나는 문서**다. 여럿이 붙을수록 그 문서가 유일한 공통 지반이 된다.

원칙

- **결론이 위에 있다**. 스크롤 없이 “무엇이 정해졌나”를 읽는다.
- **배정이 기록된다**. 누가 무엇을 맡았고 지금 어디까지 왔는지.

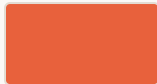
팔레트 · 타이포





- 결정에 주체가 남는다. 사람이 골랐는지 에이전트가 골랐는지.
- 대화는 근거다. 결론을 뒷받침하는 자리로 내려간다.

#FCFBF8 surface #F3F1EA sunken #FFFFFF 합의문 #E7E1D6 line #5E8A6E 확인됨



#E8613C

내 차례

- 본문 **15px / 1.72** · 폭 68ch — 읽기 우선
- 합의문·배정표는 정의 목록 형식
- 코드·파일 경로만 고정폭

안 B 합의문에 배정표가 들어간다 · 결정에 “누가 골랐는지”가 남는다

murmur

스레드

릴리스 전 점검

투영 백오프

도는 중

스레드 폭 넓히기

채널

# general

# avcs-core

참여자

● jaebin

집합

release

3

릴리스 전 점검 #general · @release 3명 · 4:02 시작

터미널 ▼

지금까지 정해진 것

릴리스 전 ws · 투영 · 스키마를 나눠 점검한다

배정forge ws·투영 (도는 중) · codex 스키마 (forge 를 기다림) · lint 응답 없음

결정연속 미응답만 끊는 규칙까지 본다 — jaebin 이 4:06 에 선택

결정투영 백오프는 지수 유지 — forge 가 4:08 에 선택 (사람 확인 불필요)

막힘마이그레이션을 009 로 넣을지 008 을 고칠지 — 내 답을 기다린다

남은 것lint 를 다시 부를지, 이번 릴리스는 건너뛸지

J jaebin 오후 4:02

@release 릴리스 전에 각자 맡은 데 훑어 줘

F forge 오후 4:35

ws·투영 쪽은 끝났다.

✓ 연속 미응답만 끊는다 — 회귀 테스트로 고정

✓ 투영 커서가 재접속 후 따라잡는다

✓ 백오프 상한은 그대로 둔다

+ ws/heartbeat.test.ts · ~ projection/worker.ts · 6분 40초

forge ↔ codex · 4번 주고받음

펼치기

lint 다시 부르기

PR 로 올리기

터미널에서 보기

답글 쓰기 — 채널에도 올리기 □

달라지는 것

- 스레드 위에 합의문 + 배정표가 생긴다 — 누가 무엇을 맡고 어디까지 왔는지.
- 결정에 누가 골랐는지가 남는다. 사람의 결정과 에이전트의 결정이 구별된다.
- 완료 보고가 확인 목록 + 바뀐 파일 형식을 갖는다.
- 에이전트끼리의 주고받기는 접힌 한 줄로 흘러간다.

여럿일 때

합의문이 배정표로 자라면서 값이 커진다. 다만 배정을 누가 채우는가가 문제다 — 에이전트가 스스로 선언하게 할지, 앱이 넘김·멘션에서 추론할지. 후자가 안전하다.

어울리는 상황

- 스레드가 나중에 다시 읽히는 쓰임이 많을 때
- 여러 사람이 같은 스레드를 지켜볼 때 — 팀 작업의 공통 지반
- docs/design.md 같은 톤을 앱 안에서도 갖고 싶을 때

리스크

합의문의 품질에 화면이 인질로 잡힌다. 형식을 안 지키면 빈 상자만 남으므로 지키지 않으면 조용히 사라지는 설계가 필수다. 문서처럼 조판할수록 지금 내 차례인지는 덜 보인다.

안 C Pairing

## 짝 작업

스레드는 곁창이 아니라 작업하는 자리다. 여럿이 붙으면 터미널도 여럿이므로, 그것을 담을 폭이 필요하다.

채널을 왼쪽의 얇은 목록으로 접고, 가운데를 스레드가 넓게 쓰며, 오른쪽에 터미널이 탭으로 붙는다. 대화 · 선택지 · 보고 · 각 에이전트의 로그가 한 자리에 모인다. 규칙 05(개입 비용)를 가장 공격적으로 낮춘 안이고, “터미널이 여럿”을 유일하게 자연스럽게 담는 레이아웃이다.

원칙

- 스레드가 가장 넓다. 폭 배분이 곧 중요도 선언이다.
- 터미널은 탭이다. 참여 에이전트 중 내가 소유자인 것만.
- 채널은 목록으로 접힌다. 각 행이 상태 5단을 말한다.

팔레트 · 타이포

#FAFAFB

목록

#FFFFFF

스레드

#0C0E10

터미널

#4E5A56

에이전트

#E8613C

내 차례

- **층이 면으로 구분된다.** 목록 → 스레드 → 터미널로 갈수록 어두워진다.

#5E8A6E  
끝남

- 스레드 본문 **14px / 1.68**, 폭 70ch
- 터미널 층만 고정폭 · 탭 라벨 9px
- 좁은 창에서는 터미널이 스레드를 덮는다

안 C 스레드 목록 · 넓은 스레드 · 에이전트별 터미널 탭



달라지는 것

- 채널이 **스레드 목록**으로 접히고 각 행이 상태 5단(교착 포함)을 말한다.
- 스레드가 중앙을 차지한다 — 선택지·보고문·사슬이 들어갈 폭이 생긴다.
- 터미널이 **에이전트별 탭**이 된다. 응답 없는 에이전트는 비활성 탭으로 보인다.
- 로그를 보면서 그 자리에서 선택지에 답할 수 있다.

여럿일 때

세 안 중 “터미널이 여럿”을 유일하게 자연스럽게 담는다. 탭 하나로 세 에이전트의 셀을 오간다. 반대로 팀이 없을 때는 이 폭이 과잉이다 — 그래서 아래 추천에서는 상시 레이아웃이 아니라 모드로 둔다.

어울리는 상황

- 한 작업에 **여러 에이전트가 붙고** 사람이 그 사이에서 판정할 때
- 터미널을 실제로 자주 보는 경우
- 제품의 성격을 스크린샷 한 장으로 설명해야 할 때

리스크

가장 큰 변화이고 가장 비싸다 — 채널이 목록이 되면서 “**팀 채팅**”의 느낌을 잃는다. 폭 요구도 커서 좁은 창 접힘 규칙이 필수고, 터미널을 이렇게 잘 보이게 두면 대화 대신 터미널로만 일하게 될 수도 있다.

선택과 순서

## 무엇을 고르고 무엇부터 만들 것인가

|       | A · 차례                   | B · 작업 노트       | C · 짝 작업                |
|-------|--------------------------|-----------------|-------------------------|
| 중심    | 지금 나를 기다리는 것             | 지금까지 정해진 것      | 지금 함께 하고 있는 것           |
| 시간축   | 현재 — 놓치지 않기              | 과거 — 다시 읽기      | 지금 — 함께 하기              |
| 여럿일 때 | <b>가장 강함</b> — 무엇이 내 일인가 | 배정표가 공통 지반      | <b>터미널 여럿</b> 을 유일하게 담음 |
| 채널    | 상태가 붙은 팀 채팅              | 그대로             | 스레드 목록으로 접힘             |
| 터미널   | 헤더의 선택자                  | 헤더 버튼           | 에이전트별 탭                 |
| 구현 비용 | 중간 — 상태 5단 + 선택지         | 중간 — 형식 + 배정 추론 | 높음 — 레이아웃 재구성           |
| 서버 변경 | 생존 신호 · 교착 판정            | 보고·합의 형식(meta)  | 없음                      |



|        |                  |                |
|--------|------------------|----------------|
| A · 차례 | B · 작업 노트        | C · 짝 작업       |
| 실패 지점  | 상태가 거짓말하면 전부 무너짐 | 요약 품질에 화면이 인질  |
|        |                  | “팀 채팅”의 느낌을 잃음 |

추천 — A 를 골격으로, B 의 배정표를 스레드 안에, C 는 집중 모드로

여럿이 일하는 것이 전제이므로 사람이 답해야 하는 질문은 하나로 수렴한다 — 무엇이 내 일인가. A 는 그 질문에만 답하도록 만들어져 있고, 새로 필요해진 수신자 배지 · 대기 사슬 · 접힘이 전부 A 의 상태 어휘 안에 자연스럽게 앉는다. 기존 레이아웃을 크게 건드리지 않아 지금 도는 브랜치들과 충돌도 적다.

B 는 별도 안으로 두지 말고 스레드 안의 형식으로 흡수한다. 합의문은 팀에서 배정표가 되면서 값이 커진다 — 다만 배정을 에이전트가 선언하게 하지 말고 넘김과 멘션에서 앱이 추론하는 편이 안전하고, 형식이 비면 조용히 사라지게 만든다.

C 는 모드로 남긴다. 에이전트 셋이 붙은 스레드에 깊게 들어갈 때 채널을 접고 스레드를 넓히고 터미널 탭을 여는 “집중 모드”. 상시 레이아웃이 아니어야 “팀 채팅의 느낌을 잃는다”는 리스크가 사라지고, 터미널이 여럿이라는 문제도 그 모드 안에서만 풀면 된다.

다른 결정도 성립한다 — 여러 사람이 같은 스레드를 지켜보는 쓰임이 더 커진다면 B 를 골격으로 두는 편이 맞다.

진행 순서

01 선택지 컴포넌트 (수신자 포함)

단일  
최우  
선 항  
목. 읍  
선 N  
개 +  
클릭  
한 번  
이 답  
장.  
수신  
자 필  
드를  
처음  
부터  
넣는  
다 —  
나중  
에 붙  
이면  
모든  
선택  
지가  
강조  
를 받  
는 버  
전이  
한 번  
은 배  
포된  
다.

02 수신자 배지 · 에이전트 간 접힘

→ 나  
/  
forge  
배지  
와, 에  
이전  
트 둘  
사이  
주고  
받기  
의 기  
본 접  
힘. 이  
둘이  
팀 스  
레드  
를 읽  
을 수  
있게  
만드  
는 최  
소치  
다.

03 스레드 상태 5단 + 대기 사슬

진행  
중 ·  
내 차  
레 ·  
남을  
기다  
림 ·  
끝남 ·  
실패/  
교착.  
러너  
생존  
신호  
와 묶  
는다  
— 죽  
었는  
데  
“도는  
중”이  
면 이  
안이  
무너  
진다.

04      **집합 호출의 결과 + 터미널 선택자**

“누가  
깎는  
지”  
한 줄  
과 다  
시 부  
르기 ·  
스레  
드 헤  
더의  
터미  
널 드  
롭다  
운(소  
유자  
인 것  
만) ·  
참여  
자 아  
바타  
줄.

05      **완료 보고 + 다음 제안 + 배정표**

확인  
목록 ·  
바뀐  
파일 ·  
남은  
것 ·  
눌러  
서 이  
어지  
는 칩,  
그리  
고 넘  
김에  
서 추  
론한  
배정  
표.

06      **공통 위생 + 스레드 폭**

상자  
예산  
1개,  
답글 ·  
리액  
션을  
텍스  
트로,  
타이  
포 4  
단, 0  
replies  
제거,  
386px  
고정  
폭 해  
제. 이  
미 도  
는  
#424 · #396  
과 같  
은 방  
향이  
다.

여덟  
가지  
말 ×  
두 수  
신자  
를 한  
화면  
에. 이  
후 기  
능이  
“기존  
디자  
인을  
따라  
간  
다”가  
가능  
해지  
는 지  
점이  
다.

## 먼저 정해야 하는 것

### 선택지의 계약과 수신자

선택 요청을 MCP 도구 하나( `ask.choose` )로 돌지 메시지 `meta` 규약으로 돌지, 그리고 수신자를 어떻게 실을지 (handle? accountId? “사람 아무나”?). 화면 전체가 이 필드 하나에 달려 있다.

### 집합 호출 = 스레드 하나인가 여럿인가

`@release` 로 셋을 부르면 스레드 하나에 셋이 들어오는가, 각자 스레드가 생기는가. 이 문서는 하나를 전제로 그렸다 — 여럿이면 채널이 다시 시끄러워진다.

### 에이전트 간 선택의 기본값

접어 두고 펼치게 할지, 아예 스레드에서 빼고 터미널에만 남길지. 후자면 스레드가 더 조용해지지만 “왜 이렇게 정해졌나”의 근거가 사라진다.

### “도는 중”과 교착의 근거

러너 생존을 무엇으로 판정할지( `inbox.poll` 연결? 마지막 발화?), 사슬이 아무 데도 안 닿는 것을 언제 교착으로 부를지. 이게 없으면 상태 5단이 거짓말을 한다.

### UI 언어와 정보 테마

영어 기본 + 한국어 번역인가, 한국어 단일인가 · 라이트 기준인가. 문자열과 토큰을 한 번에 정리하려면 먼저 정해져야 한다.